

11.4 Simple Text Output Protocol

The Simple Text Output protocol defines the minimum requirements for a text-based *ConOut* device. The EFI specification requires that the `EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL` support the same languages as the corresponding `EFI_SIMPLE_TEXT_INPUT_PROTOCOL`.

EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL

Summary

This protocol is used to control text-based output devices.

GUID

```
#define EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL_GUID \
    {0x387477c2, 0x69c7, 0x1d12, 0x9e, 0x39, 0x00, 0x00, 0, \
     0x00, 0x00, 0x12, 0x30}
```

Protocol Interface Structure

```
typedef struct _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL {
    EFI_TEXT_RESET      Reset;
    EFI_TEXT_STRING      OutputString;
    EFI_TEXT_TEST_STRING TestStrings;
    EFI_TEXT_QUERY_MODE  QueryMode;
    EFI_TEXT_SET_MODE     SetMode;
    EFI_TEXT_SET_ATTRIBUTE SetAttribute;
    EFI_TEXT_CLEAR_SCREEN ClearScreen;
    EFI_TEXT_SET_CURSOR_POSITION SetCursorPosition;
    EFI_TEXT_ENABLE_CURSOR EnableCursor;
} EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL;
```

Figure 1: Interface structure for UEFI Simple Text Output Protocol

(depending on EFI implementation). Because *ConOut* pointer comes as fifth element in *EFI System Table* (please note that we count from zero onwards), so for IA32 EFI architecture, offset will be calculated as $5 * 4 = 20$ bytes – we multiply with 4, as all pointers are 32-bit, i.e., consist of 4 bytes.

For x64 UEFI architecture, it comes as $5 * 8 = 40$ bytes, because of nature of 64-bit pointers, that are sized as 8 bytes.

So, total offsets for IA32 EFI: $A + B = 24 + 20 = 44 = 2C$ (hex). For x64 UEFI: $A + B = 24 + 40 = 64 = 40$ (hex).

Step 3: Pointers for calling function *EFI Simple Text Output Protocol* are stored in *EFI_SYSTEM_TABLE.ConOut* interface unit, the address of which we've got in Step 2.

We are interested in the function that will output the string (*OutputString*) that comes second in the interface unit. Its number is 1, if we count from zero.

In order to call this function, you should execute the 32-bit intersegment call (NEAR CALL 32) at the address of the first (we count from zero, don't forget) element from this interface unit (IA32 EFI). Since the size of one element is 32 bits (4 bytes), then the first element is a 32-bit word at offset 4.

For the x64 UEFI environment, the logic is exactly the same, just twice as long; instead of 4 bytes, it is 8 bytes, and so on.

Step 4: The function that will output a string expects two input parameters as follows.

For IA32 EFI: The first parameter is a pointer to interface block *Simple Text Output Protocol* (32-bit word in stack at offset [ESP+0]), and the second parameter is a pointer to the text string in UNICODE16 format (32-bit word in stack at offset [ESP+4]).

For x64 EFI: The first parameter is a pointer to the interface block *Simple Text Output Protocol* (64-bit word in RCX register), and the second parameter is a pointer to the text string in UNICODE16 (64-bit word stored in RDX register).

```
format pe64 dll efi
entry main
;.....;
section '.text' code executable readable main:
```

```
--- EFI application must save registers ---
push    rbx rcx rdx rsi rdi rbp
push    r8 r9 r10 r11 r12 r13 r14 r15
mov     [EFI_Handle],rcx    ; RCX=Handle from parent
mov     [EFI_Table],rdx     ; RDX=Table addr. from parent
cld

--- Output message ---
mov     rdx,Hello_String
call    Output_String

--- Wait for key press ---
Wait_Key: call    Input_Check_Key
          jc     Wait_Key    ; Wait if key not ready

--- Exit, EFI application must restore registers ---
pop     r15 r14 r13 r12 r11 r10 r9 r8
pop     rbp rdi rsi rdx rcx rbx
xor     rax,rax             ; RAX=EFI_STATUS
ret     ; Simple form of
```

termination

Alright, the pure theory part is now over. It's time to grab Flat Assembler and compile our two simple programs, so that we can execute them later in the real world environment – a UEFI-based desktop or laptop, or alternatively, in a UEFI-compatible virtual test lab with the QEMU-emulator. The latter could be launched with a specially tailored firmware image and *EFI Shell* incorporated inside (see References at the end of this article).

```
anton@dell:~/src/fasm$ ./fasm HELLO32.ASH
flat assembler  version 1.70.03 (16384 kilobytes memory)
3 passes, 2048 bytes.
```

```
anton@dell:~/src/fasm$
anton@dell:~/src/fasm$ ./fasm HELLO64.ASH
flat assembler  version 1.70.03 (16384 kilobytes memory)
4 passes, 2048 bytes.
```

The QEMU package that comes shipped by default with Ubuntu 13.10 isn't that useful for UEFI testlab, so we had to recompile it from sources: `./configure --prefix=/opt/qemu-1.7.0/ --target-list=i386-softmmu,x86_64-softmmu,i386-linux-user,x86_64-linux-user` (Note: There are NO spaces between commas in the command line)

Then please rename the BIOS- and Video-ROM files from the firmware archive *OVMF-X64-r11337-alpha.zip* as *bios.bin* and *vgabios-cirrus.bin* respectively, prepare the hard-drive image where you should place UEFI applications for a test, and run the emulator as follows:

```
$ /opt/qemu-1.7.0/bin/qemu-system-x86_64 -L . ./HD-Image.bin
```

Continued on page No 85...